

Finanzas en R — Desarrollo Asistido por IA

Positron, Claude Code y flujo de trabajo moderno

Sebastián Egaña Santibáñez (segana@fen.uchile.cl)

Nicolás Leiva Díaz (nleivad@fen.uchile.cl)

2026-09-07

Descargas

Nota Importante: Windows vs macOS/Linux

En este notebook encontrarás comandos de terminal. **Positron soporta ambos:**

- **Windows:** Usa **PowerShell** (por defecto en Positron)
- **macOS/Linux:** Usa **Bash** (por defecto)

Algunos comandos son **cross-platform** (funcionan igual en ambos): - `npm install -g ...` - `npx @anthropic-ai/claude-code ...` - `mistral-code ...`

Otros **solo existen en Bash** y necesitan equivalente en PowerShell: - `touch archivo.R` → `$null | Out-File archivo.R` (PowerShell)

En este notebook, cuando haya diferencias, mostraré ambos comandos. Si usas **Bash en Windows** (Git Bash, WSL), puedes usar los comandos Linux directamente.

Enlaces del profesor

<https://segana.netlify.app>

<https://github.com/sebaegana>

<https://www.linkedin.com/in/sebastian-egana-santibanez/>

Introducción: ¿Por qué IA en el desarrollo de análisis financiero?

En las finanzas cuantitativas modernas, **la velocidad y la precisión al codificar son críticas**. Enfrentas tres problemas recurrentes:

1. **Código boilerplate**: cargar datos, limpiarlos, preparar transformaciones básicas consume tiempo.
2. **Exploración de datos**: necesitas probar múltiples enfoques antes de encontrar el correcto.
3. **Documentación y comunicación**: traducir tu análisis a reportes reproducibles lleva tiempo extra.

Las **herramientas de IA generativa** resuelven esto. En este curso usaremos: - **Claude Code** (recomendado, pero requiere cuenta pagada) - **Mistral Vibe** (alternativa gratuita, sin tarjeta de crédito)

- **Generación rápida de código**: describe qué quieres (ej: “carga datos de una BD, calcula retornos diarios”) y obtienes código listo en segundos.
- **Debugging interactivo**: pregunta “¿por qué falla esto?” y recibe explicaciones + soluciones.
- **Documentación automática**: pide que explique y comente el código que genera.
- **Iteración ágil**: prueba ideas financieras (simulaciones, backtests, valuaciones) sin reescribir desde cero cada vez.

En este curso

Usaremos **herramientas de IA generativa** (Claude Code o Mistral Vibe) dentro de **Positron** (nuevo IDE para R/Python) para desarrollar análisis financieros reproducibles.

La meta: que escribas **menos código boilerplate** y dediques más tiempo a **pensar en finanzas**.

Tú eliges la herramienta que mejor se adapte a tu situación: - Con presupuesto → Claude Code (superior en finanzas) - Sin presupuesto → Mistral Vibe (gratis, muy bueno para aprender)

¿Qué es Positron?

Positron es una IDE moderna y de código abierto para R y Python, desarrollada por Posit Software (los creadores de RStudio). A diferencia de RStudio, Positron está construida sobre los mismos fundamentos que Visual Studio Code, lo que significa:

Ventajas de Positron frente a RStudio

Aspecto	RStudio	Positron
Base	Qt (propietaria)	VS Code (abierta, extensible)
Soporte IA nativo	No	Sí (Claude, GitHub Copilot, etc.)
Integración Python	Limitada	Primera clase
Extensiones	Pocas	Acceso a 20k+ extensiones de VS Code
Performance	Bueno	Muy bueno (más ligero)
Curva de aprendizaje	Baja	Baja (si usaste VS Code o RStudio)

Instalación de Positron

1. Ve a <https://positron.posit.co/>
2. Descarga la versión para tu SO (Windows, macOS, Linux).
3. Instala normalmente (en Windows: .exe, en macOS: .dmg).
4. Al abrir Positron, detectará automáticamente tu instalación de R (si está en PATH).

Verifica la instalación: - Abre Positron. - Ve a Terminal → New Terminal. - Ejecuta `R --version` (debería mostrar la versión de R).

Asistentes de IA en la Terminal

Realidad importante: Los modelos de IA de alta calidad requieren API keys pagadas.

Opción Recomendada: Claude Code

Claude Code es una herramienta CLI que conecta Positron con **Claude** (modelo de IA de Anthropic). Te permite interactuar con Claude directamente en la terminal de Positron, pidiendo que genere, explique, depure y documente código R.

Ventajas: - Muy preciso en finanzas y R - Excelente para debugging y explicaciones - Mejor relación costo-beneficio

Costo: Requiere API key pagada (pero tiene límites gratuitos para empezar)

Instalación y Configuración

Instalación

Todos necesitan **Node.js 18+**. Luego elige una de las dos opciones.

Paso 0: Instala Node.js (Obligatorio)

1. Descarga desde <https://nodejs.org/> (versión LTS recomendada)
2. Instala normalmente
3. Verifica en terminal:

```
node --version
npm --version
```

Opción 1: Claude Code (Si tienes cuenta pagada)

1. Crea cuenta y obtén API key:
 - Ve a <https://console.anthropic.com/>
 - Crea una cuenta e ingresa tarjeta de crédito
 - Genera una clave API (aparece como `sk-ant-...`)

2. Instala Claude Code:

```
npm install -g @anthropic-ai/npx @anthropic-ai/claude-code
```

3. Configura API key (varía por SO):

Windows (PowerShell):

```
$env:ANTHROPIC_API_KEY="sk-ant-tu-clave-aqui"
```

macOS/Linux (Bash):

```
export ANTHROPIC_API_KEY="sk-ant-tu-clave-aqui"
```

Para hacerlo permanente, agrega a ~/.bashrc, ~/.zshrc o equivalente de Windows.

4. Verifica:

```
npx @anthropic-ai/claude-code --version
```

Opción 2: Mistral Vibe (Gratis, sin tarjeta de crédito)

1. No necesitas cuenta de pago:

- Mistral Vibe es completamente gratuito
- Modelos abiertos, sin límites de uso

2. Instala Mistral Vibe:

```
npm install -g @mistralai/mistral-code
```

3. Verifica:

```
mistral-code --version
```

4. Primer uso (no requiere configuración de API):

```
mistral-code "explica qué es R"
```

¿Cuál Elegir?

Criterio	Claude Code	Mistral Vibe
Costo	Pagado	Gratis
Precisión en Finanzas	Muy alta	Buena
Facilidad de uso	Muy fácil	Muy fácil
Mejor para debugging		
Mejor para aprender R		

Recomendación: Si puedes pagar, usa Claude Code. Si no, Mistral Vibe es una excelente alternativa.

Troubleshooting: ¿No funciona Claude Code?

Error: “El término ‘npx @anthropic-ai/claude-code’ no se reconoce...”

Soluciones:

1. Usa **npx** correctamente:

- npx descarga y ejecuta el paquete automáticamente
- No necesita estar en PATH
- Ejemplo correcto:

```
npx @anthropic-ai/claude-code --file archivo.R "prompt aquí"
```

2. Verifica que Node.js esté instalado:

```
node --version  
npm --version
```

3. Reinicia PowerShell/Terminal:

- Cierra la terminal actual
- Abre una NUEVA terminal en Positron (Ctrl+Shift+‘)

4. Asegúrate de tener API key configurada:

```
$env:ANTHROPIC_API_KEY="sk-ant-tu-clave-aqui"
```

5. Si nada funciona, usa Mistral Vibe (gratis):

```
npx @mistralai/mistral-code --file archivo.R "prompt aquí"
```


Primer ejercicio: “Hello World” con IA

Enfoque tradicional (manual)

En R, el primer ejercicio es imprimir un saludo. Lo harías así:

```
print("Hola mundo")
```

```
[1] "Hola mundo"
```

Luego, exploras variables:

```
x <- 2
y <- 5
saludo <- "Finanzas en R"

print(paste(x, y, saludo, sep = " "))
```

```
[1] "2 5 Finanzas en R"
```

Tiempo invertido: ~5 minutos. **Repetitividad:** tienes que recordar la sintaxis `print()`, `paste()`, la asignación con `<-`.

Enfoque asistido por IA (Dentro de Claude Code o Mistral Code)

Pasos:

1. **Abre Positron** y crea un archivo `hello_world.R`:

- Usa `Ctrl+N` o `File → New File`
- Guarda como `hello_world.R`

2. **Abre Claude Code o Mistral Code** en la terminal:

```
claude-code # o mistral-code (si ya está instalado)
```

3. **Copia y pega este prompt directamente en la terminal interactiva:**

Crea un script R que imprima "Hola mundo" y luego prueba con dos variables `x=2`, `y=5` y una cadena, imprimiendo todo junto. El resultado debería mostrar algo como: `2 5 Finanzas en R`

Listo: Claude/Mistral generará el código, lo explica, y tú lo copias al archivo.

Resultado: Claude genera algo como:

```
# Imprimir saludo
print("Hola mundo")

# Definir variables
x <- 2
y <- 5
saludo <- "Finanzas en R"

# Imprimir junto
print(paste(x, y, saludo, sep = " "))
```

Pero **además** te muestra una explicación:

*“Cree un script básico que imprime un saludo. Usé **print()** para la salida, **<-** para asignar valores, y **paste()** para concatenar. El parámetro **sep** controla el separador entre elementos.”*

Ventaja: no solo obtienes el código, sino una **explicación instantánea**. Así aprendes mientras usas.

Buenas prácticas de prompting para R y finanzas

Cuando estés dentro de Claude Code o Mistral Code (terminal interactiva), copia y pega estos prompts de ejemplo:

Regla 1: Sé explícito sobre el contexto

Mal:

Genera código para analizar datos

Bien:

Genera un script R que cargue un CSV de retornos diarios, calcule estadísticas descriptivas (media, desviación estándar, min, max) y genere un gráfico con ggplot2

Regla 2: Pide reproducibilidad

Mal:

Carga datos de una base de datos

Bien:

Escribe un script R que se conecte a SQLite (usa el paquete DBI y RSQLite), lea una tabla 'precios' y muestre las primeras 10 filas

Regla 3: Especifica librerías que usas

Mal:

Limpia datos

Bien:

Usa tidyverse para limpiar un dataframe:

- Elimina filas con NAs
- Convierte fechas a formato Date
- Filtra observaciones posteriores a 2020

Regla 4: Pide explicaciones paso a paso

Explica qué hace cada línea de este script, paso a paso:
[pega aquí el código]

Regla 5: Itera sobre errores

Si el código genera un error, simplemente pega el error:

El script me da este error:
[pega aquí el error]

¿Cuál es el problema y cómo lo arreglo?

Ejercicio práctico integrador: Mini-análisis financiero con IA

Ahora harás un análisis financiero simple guiado por Claude Code.

Escenario

Tienes un archivo CSV con retornos diarios de una acción (o lo simularemos). Quieres: 1. Cargar los datos. 2. Calcular retornos acumulativos. 3. Generar un gráfico. 4. Calcular estadísticas de riesgo (volatilidad, Sharpe ratio).

Paso 1: Crea el archivo de datos simulados

En Claude Code o Mistral Code, copia y pega este prompt:

Crea un script R que simule 252 retornos diarios (1 año) de una acción, con media 0.05% y desviación estándar 1.5%.

Guarda el resultado en un CSV llamado 'retornos.csv' con columnas 'fecha' (fechas consecutivas desde hoy) y 'retorno'.

Claude generará algo como:

```
library(tidyverse)

-- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
v dplyr      1.1.4      v readr      2.1.5
v forcats    1.0.0      v stringr    1.5.1
v ggplot2    3.5.2      v tibble     3.3.0
v lubridate  1.9.4      v tidyr      1.3.1
v purrr      1.0.4

-- Conflicts ----- tidyverse_conflicts() --
x dplyr::filter() masks stats::filter()
x dplyr::lag()     masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become

set.seed(123)
n_dias <- 252

datos <- tibble(
```

```

    fecha = seq(Sys.Date(), by = "day", length.out = n_dias),
    retorno = rnorm(n_dias, mean = 0.0005, sd = 0.015)
)

write_csv(datos, "retornos.csv")

```

Copia este código a un nuevo archivo `simular_datos.R` en Positron y ejecútalo (Ctrl+Enter o Cmd+Enter).

Paso 2: Análisis con IA

En Claude Code o Mistral Code, copia y pega este prompt:

Lee el archivo `retornos.csv`,
 calcula los retornos acumulativos (usa `cumprod(1 + retorno) - 1`),
 grafica los retornos acumulativos con `ggplot2` en función de la fecha,
 calcula la media, desviación estándar y el ratio de Sharpe
 (asumir tasa libre de riesgo = 0),
 e imprime estos estadísticos.

Claude generará:

```

library(tidyverse)

# Leer datos
datos <- read_csv("retornos.csv")

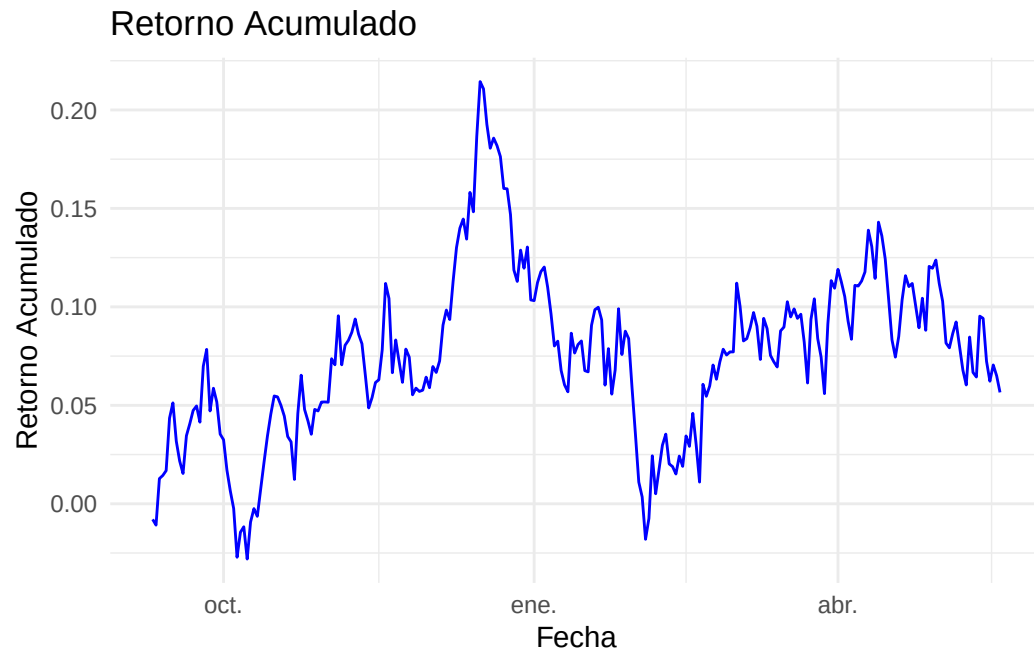
Rows: 252 Columns: 2
-- Column specification -----
Delimiter: ","
dbl  (1): retorno
date  (1): fecha

i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this message.

# Calcular retornos acumulativos
datos_acum <- datos %>%
  mutate(retorno_acumulado = cumprod(1 + retorno) - 1)

# Gráfico
ggplot(datos_acum, aes(x = fecha, y = retorno_acumulado)) +
  geom_line(color = "blue") +
  labs(title = "Retorno Acumulado",
       x = "Fecha",
       y = "Retorno Acumulado") +
  theme_minimal()

```



```
# Estadísticos
media_retorno <- mean(datos$retorno)
sd_retorno <- sd(datos$retorno)
sharpe_ratio <- (media_retorno * 252) / (sd_retorno * sqrt(252))

cat("Media de retornos:", media_retorno, "\n")
```

Media de retornos: 0.0003168098

```
cat("Volatilidad anualizada:", sd_retorno * sqrt(252), "\n")
```

Volatilidad anualizada: 0.2236562

```
cat("Sharpe Ratio:", sharpe_ratio, "\n")
```

Sharpe Ratio: 0.356959

Paso 3: Debugging interactivo

Si algo falla, simplemente pega el error en Claude Code/Mistral Code:

El script da este error:

Error: object 'retorno_acumulado' not found

¿Qué está mal y cómo lo arreglo?

Paso 4: Documentación

Pide que agregue comentarios:

Agrega comentarios en español explicando qué hace cada sección del script,

incluyendo qué paquetes usamos y por qué.
Aquí está el código:
[pega el código aquí]

Ciclo de trabajo recomendado

El flujo ideal en un análisis financiero con IA es:

Idea financiera

↓

Describe qué quieres a Claude Code

↓

Claude genera código

↓

Ejecuta en Positron (prueba rápida)

↓

¿Funciona?

No → Pídele a Claude que lo depure

Sí → ¿Resultado coherente?

No → Ajusta el prompt, reitera

Sí → Integra en tu análisis principal

Ventaja clave: en lugar de pasar 2 horas escribiendo código, pasas 20 minutos iterando con Claude. El ahorro acumulado es enorme en un semestre.

Limitaciones y ética

Lo que Claude Code no es

- No reemplaza tu comprensión de R o finanzas. Usa Claude para **acelerar**, no para abandonar el aprendizaje.
- No es 100 % perfecto. El código generado puede tener bugs. **Siempre revisa antes de usar en datos reales.**
- No trabaja sin internet (necesita conectarse a Anthropic).

Buenas prácticas éticas

1. **Entiende el código que usas:** si Claude genera algo que no comprendes, pregunta o estudia hasta que lo entiendas.
 2. **Verifica resultados financieros:** si estás haciendo un backtest o valuación, valida los números con métodos alternativos.
 3. **Credibilidad en reportes:** en un informe profesional, documenta que usaste IA pero que validaste los resultados.
-

Actividad para el estudiante

Objetivo

Replicar el flujo de trabajo aprendido con un **dataset distinto**: usa datos financieros reales (ej: puedes obtenerlos de Yahoo Finance, FRED, o una BD local).

Instrucciones

1. **Obtén o simula datos**: retornos de 3 acciones (ej: Apple, Google, Microsoft) durante 1 año.

Opción A (simulado con Claude):

```
npx @anthropic-ai/claude-code "simula retornos diarios de 3 acciones distintas (252 días)"
```

Opción B (datos reales): descarga desde <https://finance.yahoo.com/>

2. **Analiza con Claude Code**:

- Carga los datos.
- Calcula retornos acumulativos por acción.
- Genera gráficos comparativos.
- Calcula correlación entre las 3 acciones.

Pídele a Claude que genere un script que haga todo esto de una vez.

3. **Entrega**:

- Script R comentado (.R).
- Resultado de la correlación (tabla numérica).
- Un gráfico (puede ser PNG exportado de Positron).
- Párrafo breve (3-5 líneas): qué aprendiste de usar Claude Code vs escribir código manual.

Rúbrica

- Script funciona sin errores: 40 %
- Usa tidyverse y ggplot2 (como el resto del curso): 20 %
- Cálculos financieros correctos: 20 %
- Documentación clara (comentarios, texto explicativo): 20 %

Evaluación de Conocimientos

Esta sección contiene 5 preguntas para evaluar tu comprensión de la clase.

 Advertencia

Contenido protegido: Las preguntas solo están disponibles para el profesor. Ingresa la contraseña para desbloquear.

Próximos pasos

En las siguientes clases: - Usaremos Claude Code para generar análisis de agregación, uniones y transformaciones de datos (Clase 02). - Haremos regresión y modelado financiero asistido por IA (Clase 03). - Construiremos simulaciones de Monte Carlo y backtests.

La meta: que al final del curso, no solo entiendas R y finanzas, sino que sepas **cómo herramientas modernas de IA aceleran tu desarrollo profesional.**

Referencias y recursos

- **Positron:** <https://positron.posit.co/>
- **Claude Code:** [@anthropic-ai/claude-code](https://github.com/anthropics/npx)
- **Anthropic API:** <https://console.anthropic.com/>
- **tidyverse:** <https://www.tidyverse.org/>
- **ggplot2:** <https://ggplot2.tidyverse.org/>
- **Finanzas cuantitativas en R:** “Quantitative Financial Analytics with R” (Córdoba Padilla)